



Algoritmos e Programação

Estruturas de Repetição, Contadores e Acumuladores

Prof. Pedro Bilar Montero
Universidade Federal de Santa Maria
Colégio Politécnico
Curso Técnico em Informática

Agenda

Revisão Rápida

Desvio Condicional: Switch-case

Laços de Repetição: Introdução

Enquanto (*while*)

Repita (*do-while*)

Para (*for*)

Contadores

Acumuladores

Sinalizador (*Flag*)

Exercícios

Aulas anteriores:

- ▶ **printf / scanf** — exibir e ler dados
- ▶ **Tipos de dados** — `int`, `float`, `char`
- ▶ **Operadores relacionais** — `==`, `!=`, `<`, `>`, `<=`, `>=`
- ▶ **Operadores lógicos** — `&&`, `||`, `!`
- ▶ **if / else if / else** — desvios condicionais

Hoje vamos aprender:

- ▶ **Switch-case** — outra forma de tomar decisões
- ▶ **Laços de repetição** — executar código várias vezes
- ▶ **Contadores e Acumuladores** — variáveis especiais

O que é o Switch-case?

Definição

Estrutura usada quando precisamos testar uma variável para **vários valores fixos e conhecidos**.

Quando usar?

- ▶ A variável pode assumir um número finito de valores
- ▶ Geralmente usado para construção de **menus**
- ▶ A variável deve ser do tipo **inteiro** ou **caractere**

Como funciona?

1. O programa testa a variável contra cada case
2. Quando encontra um valor igual, executa aquele bloco
3. O break faz o programa sair da estrutura
4. O default executa se nenhum caso bater

Comparação: if-else-if vs switch-case

Com if-else-if:

```
1  if (variavel == 1) {  
2      <comandos 1>  
3  } else if (variavel == 2) {  
4      <comandos 2>  
5  } else if (variavel == 3) {  
6      <comandos n>  
7  } else {  
8      <comando padrao>  
9  }
```

Com switch-case:

```
1  switch (variavel) {  
2      case 1:  
3          <comandos 1>  
4          break;  
5      case 2:  
6          <comandos 2>  
7          break;  
8      case 3:  
9          <comandos n>  
10         break;  
11     default:  
12         <comando padrao>  
13         break;  
14 }
```

Switch-case: Regras Importantes

- ▶ A variável testada deve ser **inteiro** (int) ou **caractere** (char)
- ▶ O break é essencial: sem ele, o programa continua executando os próximos cases
- ▶ O default funciona como o else: executa se nenhum case combinar
- ▶ O default é **opcional**

Atenção — Sem break!

Se esquecer o break, o programa *cai* para o próximo case e executa também, mesmo que o valor não bata. Isso raramente é o que você quer!

Exercício: Calculadora com Switch-case (Enunciado)

Enunciado

Escreva um programa que leia dois números e uma opção de operação (1=Adição, 2=Subtração, 3=Multiplicação, 4=Divisão) e exiba o resultado.

Requisitos:

- ▶ Mostrar um menu de operações para o usuário
- ▶ Ler dois números reais
- ▶ Usar `switch-case` para selecionar a operação
- ▶ Tratar divisão por zero
- ▶ Exibir mensagem para opção inválida

Exercício: Calculadora com Switch-case (Resolução 1/2)

```
1  #include <stdio.h>
2
3  int main() {
4      int operacao;
5      float num1, num2;
6
7      printf("1-Adicao  2-Subtracao\n");
8      printf("3-Multiplicacao  4-Divisao\n");
9      printf("Escolha a operacao: ");
10     scanf("%d", &operacao);
11
12     printf("Digite os dois numeros: ");
13     scanf("%f %f", &num1, &num2);
14     switch (operacao) {
15         case 1:
16             printf("Resultado: %.2f\n", num1 + num2);
17             break;
```


Exercício: Calculadora com Switch-case (Resolução 2/2)

```
1         case 2:
2             printf("Resultado: %.2f\n", num1 - num2);
3             break;
4         case 3:
5             printf("Resultado: %.2f\n", num1 * num2);
6             break;
7         case 4:
8             if (num2 != 0)
9                 printf("Resultado: %.2f\n", num1 / num2);
10            else
11                printf("Erro: divisao por zero!\n");
12            break;
13        default:
14            printf("Opcao invalida.\n");
15    }
16
17    return 0;
```

Por que precisamos de laços?

Imagine imprimir “Ola!” cinco vezes:

```
1 int main() {  
2     printf("Ola!\n");  
3     printf("Ola!\n");  
4     printf("Ola!\n");  
5     printf("Ola!\n");  
6     printf("Ola!\n");  
7 }
```

E se fossem 100 vezes? Ou 1000?

Repetir código manualmente é impraticável. Precisamos de uma forma de dizer ao computador: “**repita isso N vezes**”.

Os 3 Laços de Repetição em C

`while` (Enquanto)

- ▶ Testa a condição **antes**
- ▶ Pode nunca executar
- ▶ Ideal quando não sabemos quantas repetições

`do-while` (Repita-enquanto)

- ▶ Testa a condição **depois**
- ▶ Executa **ao menos 1 vez**
- ▶ Ideal para menus e validações

`for` (Para)

- ▶ Testa a condição **antes**
- ▶ Contador embutido
- ▶ Ideal quando sabemos o número exato de repetições

Regra geral

Todos fazem a mesma coisa — a diferença é onde e como testam a condição de parada.

Enquanto (*while*): Conceito

Definição

Executa um bloco de código **enquanto** uma condição for verdadeira. A condição é verificada **antes** de cada execução.

Características:

- ▶ Se a condição for **falsa desde o início**, o bloco **nunca é executado**
- ▶ Diferente do `if`: o `while` pode executar o mesmo trecho **múltiplas vezes**
- ▶ É necessário atualizar a variável da condição dentro do laço, senão ele nunca para

Enquanto (*while*): Sintaxe

```
1 while (<condicao>) {  
2     <linha de instrucao 1>  
3     <linha de instrucao 2>  
4     <linha de instrucao 3>  
5     ...  
6 }
```

Fluxo de execução:

1. Avalia a <condicao>
2. Se **verdadeira**: executa o bloco, volta ao passo 1
3. Se **falsa**: pula o bloco e continua o programa

While: Antes e Depois (1/3)

Antes — sem laço (código repetido):

```
1 int main() {  
2     printf("Algoritmos\n");  
3     printf("Algoritmos\n");  
4     printf("Algoritmos\n");  
5 }
```

↓ Trechos iguais! Podemos escrever apenas 1 vez

While: Antes e Depois (2/3)

Agora — com while:

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5
6      while (i < 3) {
7          printf("Algoritmos\n");
8          i = i + 1;
9      }
10 }
```

A variável *i* controla quantas vezes o laço executa.

i = *i* + 1 garante que o laço vai parar.

While: Antes e Depois (3/3) — Passo a Passo

Como o computador executa o laço:

Passo	Valor de i	i < 3?	O que acontece
1	0	Sim	Imprime “Algoritmos”, i vira 1
2	1	Sim	Imprime “Algoritmos”, i vira 2
3	2	Sim	Imprime “Algoritmos”, i vira 3
4	3	Não	Sai do laço!

Resultado na tela: “Algoritmos” aparece **3 vezes**.

While: Cuidado com Loop Infinito!

Loop Infinito

Se a condição **nunca ficar falsa**, o programa fica preso para sempre. Isso se chama **loop infinito**.

Exemplo de loop infinito (ERRADO):

```
1  int i = 0;
2  while (i < 3) {
3      printf("Algoritmos\n");
4      // esqueceu de incrementar i !
5  }
```

Regra: sempre atualize a variável de controle

Garanta que em algum momento a condição do while ficará falsa. Caso contrário, o programa nunca termina.

While: Exemplo — Números Pares (enunciado)

Problema

Mostrar na tela todos os números inteiros positivos pares menores que 10.

Pensando a solução:

- ▶ Pares menores que 10: 2, 4, 6, 8
- ▶ Podemos começar com `num = 2`
- ▶ A cada passo, somamos 2 (`num = num + 2`)
- ▶ Paramos quando `num` chegar a 10

Condição do while: `num < 10`

While: Exemplo — Números Pares (solução)

```
1  #include <stdio.h>
2
3  int main() {
4      int num = 2; // primeiro par positivo
5
6      while (num < 10) {
7          printf("%d ", num);
8          num = num + 2; // proximo par
9      }
10
11     printf("\n");
12     return 0;
13 }
```

Saída: 2 4 6 8

Teste de mesa: num=2 (imprime) → num=4 (imprime) → num=6 (imprime) → num=8 (imprime) → num=10 → **10 < 10 é Falso** → **para!**

Enunciado

Leia números inteiros até o usuário digitar 0. Ao final, mostre quantos números positivos foram digitados.

Requisitos:

- ▶ Usar `while` para repetir a leitura
- ▶ Usar um contador para positivos
- ▶ Encerrar quando o valor lido for 0
- ▶ Exibir o total no final

Repita (*do-while*): Conceito

Definição

Executa o bloco de código **primeiro**, e só depois verifica a condição. Se for verdadeira, repete.

Características:

- ▶ O bloco é executado **pelo menos 1 vez**, independentemente da condição
- ▶ A condição é verificada **após** cada execução
- ▶ Ideal para menus: mostra o menu antes de perguntar se o usuário quer sair

Diferença chave em relação ao while

No **while**, se a condição já for falsa de início, o bloco **nunca executa**.

No **do-while**, o bloco **sempre executa ao menos uma vez**.

Repita (*do-while*): Sintaxe

```
1  do {  
2      <linha de instrucao 1>  
3      <linha de instrucao 2>  
4      <linha de instrucao 3>  
5      ...  
6  } while (<condicao>;
```

Fluxo de execução:

1. Executa o bloco
2. Avalia a <condicao>
3. Se **verdadeira**: volta ao passo 1
4. Se **falsa**: continua o programa

Note o ponto e vírgula **após** o while(<condicao>). Não esqueça!

Do-while: Exemplo e Comparação (1/2)

Mesmo exemplo com do-while:

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5
6      do {
7          printf("Algoritmos\n");
8          i = i + 1;
9      } while (i < 3);
10 }
```

Resultado: “Algoritmos” impresso **3 vezes**. O mesmo resultado do while, pois $i = 0$ já satisfazia a condição.

Do-while: Exemplo e Comparação (2/2)

Quando a diferença aparece — condição falsa desde o início:

Com while:

```
1  int i = 5;
2  while (i < 3) {
3      printf("Executa!\n");
4      i++;
5  }
6  // Não imprime nada!
```

O while não executa nada ($5 < 3$ é falso).

O do-while executa uma vez antes de checar a condição.

Com do-while:

```
1  int i = 5;
2  do {
3      printf("Executa!\n");
4      i++;
5  } while (i < 3);
6  // Imprime 1 vez!
```

Exercício: Do-while

Enunciado

Faça um menu com as opções 1-Somar, 2-Subtrair e 0-Sair. O menu deve aparecer pelo menos uma vez e repetir até o usuário escolher 0.

Requisitos:

- ▶ Usar `do-while`
- ▶ Ler a opção em cada repetição
- ▶ Exibir mensagem para opção inválida
- ▶ Finalizar apenas com a opção 0

Para (*for*): Conceito

Definição

Utilizado quando o programador sabe **exatamente quantas vezes** um conjunto de instruções deve ser executado. Possui contador embutido na própria estrutura.

Características:

- ▶ Inicialização, condição e incremento ficam **na mesma linha**
- ▶ Código mais compacto para situações de contagem
- ▶ A condição é verificada **antes** de cada iteração
- ▶ Funciona exatamente como o while, mas com sintaxe mais conveniente para contadores

Para (*for*): Sintaxe Explicada

```
1  for (inicializacao; condicao; incremento) {  
2      <linha de instrucao 1>  
3      <linha de instrucao 2>  
4      ...  
5  }
```

Parte	Exemplo	Quando executa
Inicialização	<code>i = 0</code>	Apenas 1x, no início
Condição	<code>i < 3</code>	Antes de cada iteração
Incremento	<code>i++</code>	Ao final de cada iteração

`i++` é a mesma coisa que `i = i + 1`.

For: Exemplo 1 — Básico (1/2)

Imprimindo “Algoritmos” 3 vezes com for:

```
1  #include <stdio.h>
2
3  int main() {
4      int i;
5      for (i = 0; i < 3; i++) {
6          printf("Algoritmos\n");
7      }
8  }
```

Compare com o while equivalente:

```
1  int i = 0;
2  while (i < 3) {
3      printf("Algoritmos\n");
4      i++;
5  }
```

For: Exemplo 1 — Básico (2/2) — Passo a Passo

Como o `for` executa:

Passo	Ação	i	$i < 3?$
Início	Inicialização: $i = 0$	0	—
1	Checa condição $\rightarrow$ executa $\rightarrow$ incrementa	$0 \rightarrow 1$	Sim
2	Checa condição $\rightarrow$ executa $\rightarrow$ incrementa	$1 \rightarrow 2$	Sim
3	Checa condição $\rightarrow$ executa $\rightarrow$ incrementa	$2 \rightarrow 3$	Sim
4	Checa condição $\rightarrow$ sai!	3	Não

For: Exemplo 2 — Números Pares

Mesmos números pares, agora com for:

```
1  #include <stdio.h>
2
3  int main() {
4      int num;
5      for (num = 2; num < 10; num += 2) {
6          printf("%d ", num);
7      }
8      printf("\n");
9      return 0;
10 }
```

Saída: 2 4 6 8

Observe o incremento

`num += 2` é igual a `num = num + 2`, ou seja, incrementa de 2 em 2 (não apenas 1). O incremento do for pode ser qualquer expressão!

Quando usar cada laço?

Laço	Use quando...	Exemplo
while	Não sabe quantas repetições serão necessárias	Ler dados até digitar -1
do-while	O bloco deve executar pelo menos 1 vez (menus, validações)	Menu de opções
for	Sabe exatamente o número de repetições	Tabuada

Na prática

Na maioria dos casos você pode usar qualquer um dos três. Escolha o que torna o código mais **legível**.

Enunciado

Escreva um programa que mostre a tabuada de um número inteiro N, de 1 até 10, usando `for`.

Requisitos:

- ▶ Ler o valor de N
- ▶ Usar `for` com contador de 1 a 10
- ▶ Mostrar no formato: `N x i = resultado`

O que é um Contador?

Definição

Uma variável usada para **contar** o número de vezes que um evento ocorre durante a execução do programa.

Exemplos de uso:

- ▶ Total de alunos lidos
- ▶ Total de alunos aprovados / reprovados
- ▶ Total de clientes em débito
- ▶ Quantas vezes o usuário errou a senha

Características:

- ▶ Sempre **inicializada com zero**
- ▶ Incrementada de **1 em 1** a cada evento

Representação em C: `cont = cont + 1` OU `cont++`

Contador: Exemplo — Alunos Aprovados (1/2)

Problema

Para uma turma de 20 alunos, contar quantos foram aprovados (média maior ou igual a 6).

```
1  int main() {
2      int aprovados, i, media;
3      aprovados = 0; // inicializa contador
4
5      for (i = 0; i < 20; i++) {
6          scanf("%d", &media);
7          if (media >= 6) {
8              aprovados++; // conta aprovado
9          }
10     }
11     printf("Aprovados: %d\n", aprovados);
12 }
```

Contador: Exemplo — Alunos Aprovados (2/2) — Teste de Mesa

O que acontece a cada iteração:

Iteração (i)	media lida	media ≥ 6 ?	aprovados
0	8	Sim	1
1	4	Não	1
2	7	Sim	2
3	5	Não	2
...	...	...	...

Ao final das 20 iterações, aprovados contém o total de alunos com média ≥ 6 .

Enunciado

Leia a idade de 15 pessoas e conte quantas têm 18 anos ou mais.

Requisitos:

- ▶ Usar laço para ler as 15 idades
- ▶ Usar um **contador** iniciado em 0
- ▶ Incrementar quando `idade >= 18`
- ▶ Mostrar o total ao final

O que é um Acumulador?

Definição

Uma variável usada para **somar (acumular)** valores ao longo da execução do programa.

Exemplos de uso:

- ▶ Total de despesas no mês
- ▶ Somatório das notas de um aluno
- ▶ Total de vendas do dia

Diferença entre Contador e Acumulador:

	Contador	Acumulador
Incremento	Sempre +1	Valor variável
Exemplo	<code>cont++</code>	<code>soma += valor</code>
Uso	Contar eventos	Somar valores

Representação: `soma = soma + valor` OU `soma += valor`

Acumulador: Exemplo — Somar Despesas (1/2)

Problema

Somar as despesas de uma pessoa até ela digitar um valor negativo para sair.

```
1  int main() {
2      float valor, despesas;
3      valor = 0;
4      despesas = 0; // inicializa acumulador
5
6      do {
7          despesas = despesas + valor; // acumula
8          printf("Digite uma despesa (-1 para sair): ");
9          scanf("%f", &valor);
10     } while (valor >= 0);
11
12     printf("Total: R$ %.2f\n", despesas);
13 }
```

Acumulador: Exemplo — Somar Despesas (2/2) — Teste de Mesa

Executando com entradas: 50.0, 30.0, 20.0, -1

Passo	valor	despesas antes	despesas depois
1	0.0 (inicial)	0.0	$0.0 + 0.0 = 0.0$
2	50.0	0.0	$0.0 + 50.0 = 50.0$
3	30.0	50.0	$50.0 + 30.0 = 80.0$
4	20.0	80.0	$80.0 + 20.0 = 100.0$
5	-1.0	—	Sai! $(-1 < 0)$

Saída: Total: R\$ 100.00

Exercício: Acumulador

Enunciado

Leia 8 valores reais e calcule a soma total e a média desses valores.

Requisitos:

- ▶ Usar um acumulador iniciado em 0
- ▶ Somar cada valor lido no acumulador
- ▶ Calcular média com $media = soma / 8$
- ▶ Exibir soma e média ao final

O que é um Sinalizador (Flag)?

Definição

Uma variável usada para **sinalizar** se um evento ocorreu durante a execução, sem importar quantas vezes.

Exemplos:

- ▶ Marcar se um número é primo
- ▶ Indicar se o usuário digitou um valor inválido
- ▶ Sinalizar se um produto foi encontrado em uma lista

Características:

- ▶ Inicializada com valor **inativo** (geralmente 0)
- ▶ Muda para valor **ativo** (1) quando o evento ocorre

Representação: $\text{flag} = 0$ (inativo) ou $\text{flag} = 1$ (ativo)

Flag e Variáveis Booleanas

Conexão de conceitos

Uma **flag** guarda um estado de **sim/não** (verdadeiro ou falso), exatamente como uma variável booleana.

- ▶ 0 representa **falso**
- ▶ 1 (ou qualquer valor diferente de zero) representa **verdadeiro**

Exemplo de equivalência:

```
1  int encontrou_negativo = 0; // falso
2
3  if (numero < 0) {
4      encontrou_negativo = 1; // verdadeiro
5  }
```

Ideia central: flag é uma forma prática de representar uma condição booleana ao longo do programa.

Flag: Exemplo — Verificar Número Primo (1/2)

Problema

Ler um número e dizer se ele é ou não primo. (Um número é primo se só é divisível por 1 e por ele mesmo.)

```
1  int main() {
2      int numero, eh_primo, i;
3      scanf("%d", &numero);
4      eh_primo = 0; // assume que eh primo (flag inativo)
5
6      for (i = 2; i < numero; i++) {
7          if (numero % i == 0) {
8              eh_primo = 1; // achou divisor: NAO e primo
9          }
10     }
```

Flag: Exemplo — Verificar Número Primo (2/2)

Continuação — exibir resultado:

```
1     if (eh_primo == 0) {
2         printf("%d e primo!\n", numero);
3     } else {
4         printf("%d nao e primo.\n", numero);
5     }
6 }
```

Como funciona:

- ▶ `eh_primo = 0`: assume primo (nenhum divisor encontrado)
- ▶ O `for` testa todos os valores de 2 até `numero-1`
- ▶ Se encontrar qualquer divisor: `eh_primo = 1` (ativa a flag)
- ▶ Ao final, a flag indica o resultado

Exercício: Flag

Enunciado

Leia 10 números inteiros e informe se pelo menos um número negativo foi digitado.

Requisitos:

- ▶ Criar uma flag iniciada com 0
- ▶ Se encontrar número negativo, ativar a flag (valor 1)
- ▶ Ao final, exibir mensagem conforme o estado da flag

Exercício 1 — Soma de Pares

Escreva um programa que calcule a soma de todos os números pares de 1 a 100 usando um laço `for`.

Exercício 2 — Médias

Leia as notas de 10 alunos e calcule: a média da turma, o total de aprovados (nota ≥ 6) e o total de reprovados.

Obrigado e bons estudos!